

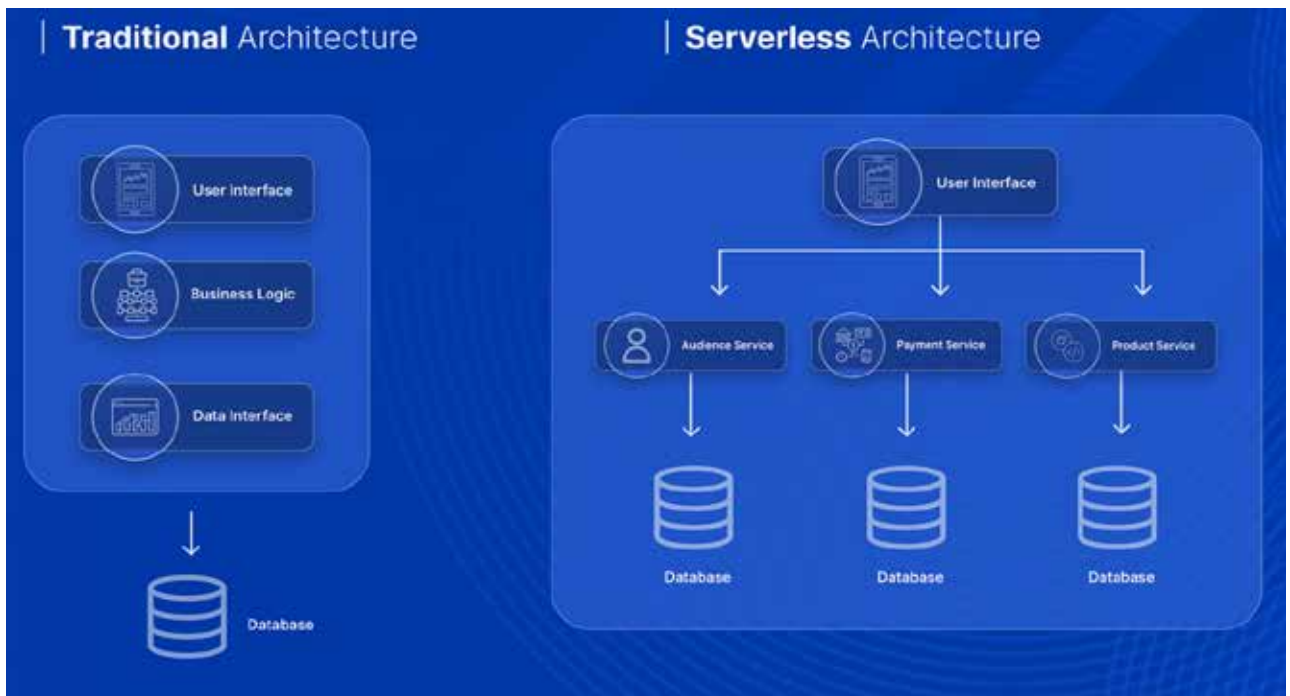


Figure 1: Traditional vs serverless computing

Feature	Traditional computing	Serverless computing
• Infrastructure management	Developers manage servers, operating systems, and networking.	Platform provider manages infrastructure.
• Resource allocation	Developers allocate resources upfront, often leading to underutilisation or overprovisioning.	Resources are allocated dynamically based on demand, optimising costs.
• Scaling	Manual scaling is required to handle increased load.	Automatic scaling ensures optimal performance without manual intervention.
• Billing	Pay for upfront costs, regardless of usage.	Pay only for the resources consumed, often based on usage metrics like execution time and memory.
• Development model	Monolithic applications are common.	Microservices architecture is well-suited for serverless.
• Deployment	Deploy applications to physical or virtual servers.	Deploy functions to the cloud platform.
• Maintenance	Ongoing maintenance is required for servers and operating systems.	Platform provider handles maintenance.
• Complexity	Higher complexity due to infrastructure management.	Lower complexity, allowing developers to focus on application logic.
• Use cases	Suitable for various applications, especially those with predictable workloads.	Ideal for event-driven applications, microservices, and applications with varying workloads.

Table 1: A comparison of traditional and serverless computing